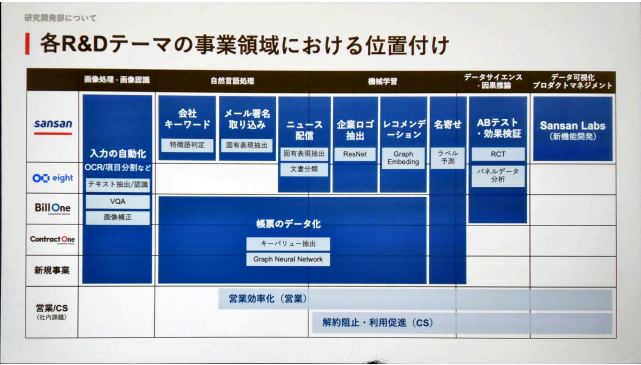


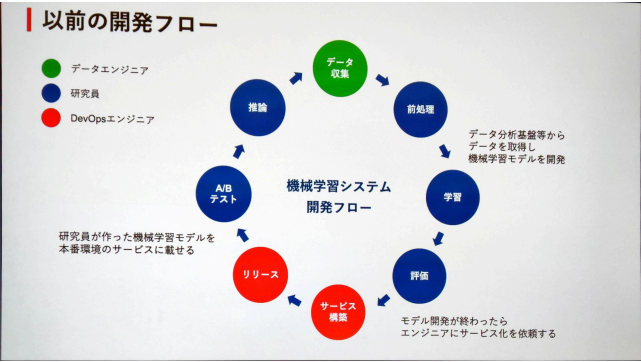
sansan k8s

2025年6月26日 13:30

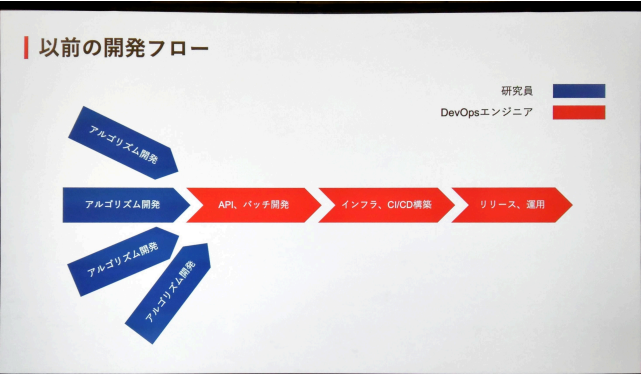
研究開発部
コア技術の提供が主業務



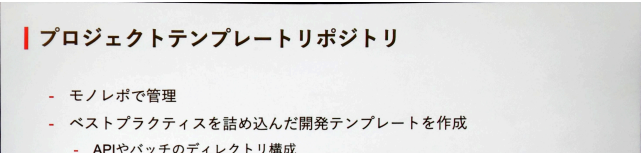
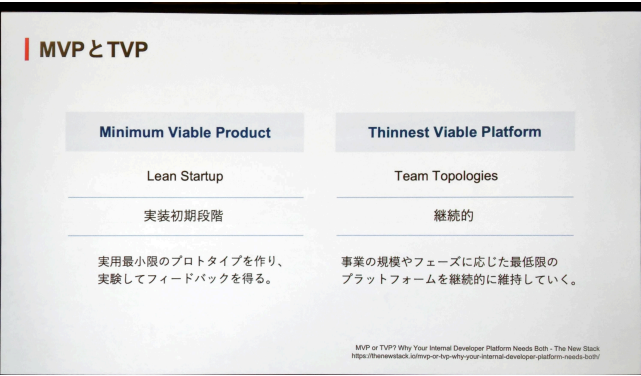
以前



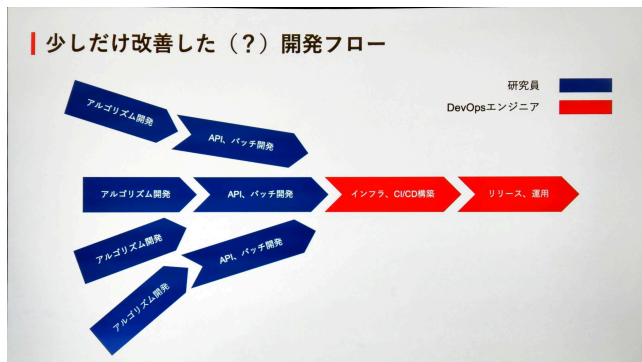
研究員とエンジニアの比率に違いがあり、ボトルネックが発生



MVPとTVPの違い



- パッケージ管理
- Dockerfile
- etc..
- Cookiecutter
 - プロジェクトテンプレートをCLIで作成できるpython製OSS



EKS

アプリケーション実行基盤の構築

Amazon Elastic Kubernetes Service (Amazon EKS)を採用

- 当初はSansan Labsのデリバリーを最適化する基盤として構築
- 顧客のスコープを広げすぎないことを意識

名前をつける

- Amazon EKSを中心とするプラットフォームを**Circuit**と命名
- 名前をつけると耳に入る頻度が上がり、認知されるようになる



テンプレートを直接編集せずに、生成されたYAMLを個別に編集するようにしてテンプレートが肥大化しないように

認知負荷の軽減

研究員にKubernetesの習得を強制したくない

Kubernetesリソースのテンプレートを作成

- アプリ、API、バッチ用のテンプレートを用意
- とにかく薄く提供する
 - テンプレートで対応できない個別カスタマイズは生成されたyamlファイルを直接編集
 - カプセル化されたWrapperを最初から作り込みすぎない

さらに手間を減らす

- UI上でボチボチするとテンプレートを使ってPRが作成される

移行基準

他環境システムのCircuit移行基準

すべてのシステムをCircuitに乗せればいいわけではない。

安定稼働しているシステムの移行判断は慎重に行う必要がある。

- なぜ移行したいのか？
- 移行することで何がうれしいのか？
 - コストが下がる？運用が楽になる？

Circuitへの移行を進めるためには、ユーザ目線で体験向上が必須である。

ユーザ体験がすべて

移行例

CI/CD環境の統一化

→デプロイ速度アップ、エンジニアの認知負荷を減らす

他環境システムのCircuit移行事例

Linux serverからの移行	
背景	成果
- Linux serverで運用（非コンテナ） - 7年以上の安定稼働	- 他システムと統一されたCI/CDによりミドルウェアの更新作業を1時間以内に改善 - リソース効率向上により、20%以上のコスト削減効果 - 専用デプロイフローを減らすことでエンジニアの認知負荷を軽減
課題	
- 非コンテナかつ、CI/CDの整備が無いためミドルウェアの更新作業に1日 - インスタンスサイズとアプリリソースに差があり、リソースの無駄が多い	

財務視点のコスト按分

会計処理が絡む

利用者視点のコスト按分

アプリ単位の最適化

なぜコスト按分が必要か？

Kubernetesの高度な柔軟性によって、コストが隠蔽されている。

財務視点の要望:

- 製品・部門ごとのコスト按分をしたい。
- 会計処理に連携可能な形式が望ましい。
- AWSコストタグに基づいた集計を行いたい。

利用者視点の要望:

- アプリ単位でコストを把握したい。
- コストの高い要因を特定して最適化したい。
- ダッシュボードでの可視化がしたい。

財務視点のコスト按分（サーバ費）

Node毎に単一NamespaceのPodのみをスケジューリングする。

NodeとなるAmazon Elastic Compute Cloud (Amazon EC2)にコスト管理タグを設定しサーバコストを管理する。

Karpenter

nodeaffinityなどで、ノードがデプロイされるノードをコントロール

財務視点のコスト按分（サーバ費）

Karpenterのリソース設定

- EC2NodeClass: Amazon EC2のタグをコスト管理タグとして設定
- NodePool: Taintの設定、Labelの設定

Podの設定

- Taintに合わせて、Toleration を設定
- NodePoolのLabelに合わせて、NodeSelector またはAffinityを指定

ログのコスト

財務視点のコスト按分（ログ費）

なぜログコストも按分するのか？

- ログは意外とコストが高く、見過ごせない存在である。
- ログコストも按分対象に統合することで正確なコストを提供する。
- 高頻度なログ出力アプリでは、サーバ費用を上回るケースもある。

利用者視点のコスト可視化

従来KubecostをAmazon EKS Addonより導入していた。情報量は十分であるが、無料版では15日間しかデータが保持できないためSplit Cost Allocation Data for Amazon EKS (SCAD) への乗り換えを行った。

Kubecost	SCAD
メリット	メリット
- EKS Addonから簡単に導入可能	- 無料で利用可能
- ダッシュボードがリッチ	- データ保持が無制限
デメリット	デメリット
- 無料版は15日間のみ保存	- 保存先 (Prometheus server, Amazon Simple Storage Service) が必要
- CURの統合が少し面倒	- Dashboardは自身で用意が必要

Kubecost | <https://www.kubecost.com/>
AWS EKS Addon Kubecost | <https://docs.aws.amazon.com/eks/latest/userguide/kubecost-monitoring-kubecost-bundles.html>
SCAD | <https://docs.aws.amazon.com/eks/latest/userguide/split-cost-allocation-data.html>

CostUsageReportに統合しないとSpotやリザーブド残すとが正しく反映されない

Image tag変更手動運用の課題

背景と手動運用の問題点

- CircuitではArgoCDによるGitOpsを採用している。
→ 全ての変更はGitHubリポジトリを経由する必要がある。
- 手動運用時の課題
 - Image Tagの書き換えミス
 - Image Tagの変更にPR作成→他者Approve→Mergeが必須である。
 - 変更の反映に数分〜半日かかることがある。
 - 開発サイクルの高速化の障害になっている。

OSS (ArgoCD Image Updater) の導入と課題

導入経緯

- 手動運用課題を解決するためにOSSの **ArgoCD Image Updater** を導入

システム数の増加により直面した技術的課題

- Amazon Elastic Container Registry (ECR) の認証処理が非効率
 - 複数スレッド認証を繰り返す問題 (認証の多重発生)

認証の多重問題解決のため、直列化した。
結果として、イメージpush後のPR作成に**最大30分**遅延

内製化したImage updaterの導入と改善効果

多数のシステムに耐えうる設計

- 認証キャッシュの効率化
 - Amazon ECRへの認証結果を一定期間キャッシュし、再認証を削減
- 非同期処理による高速化
 - Amazon ECRへの変更検知にAmazon EventBridge → Amazon Simple Queue Service (Amazon SQS) を利用しイベント駆動を実現

導入後の改善効果

	手動運用	ArgoCD Image Updater	内製 Image Updater
Image tag更新の所要時間	数分から半日程度	3~30分程度	約1分
開発者の作業負荷	PRの作成、他者Approve	自己完結可能	自己完結可能

Sansan Tech Blog (ArgoCD Image Updaterから内製ツールに移行した話) | <https://buildersbox.sansan.com/entry/20241116/110000>

構成図

